

Name: Sophia Vinh-Reyes

Student ID#: 7334339093

HW # 1

PROBLEM #1: course Policies

- a. Students have 3 excused lab sessions and must email their TA and professor
- b. Students will not receive another excused absence unless USC campus support indicates that a valid emergency has occurred
- c. false: syllabus states "we CANNOT guarantee taking the exam at an alternate time," which is different from saying "no alternative time will ever be provided"
- d. I agree that struggling through a programming problem can be important part of learning because it forces me to understand why my code works instead of only trying to reach the correct answer. I also agree that relying too much on AI could make homework easier in the short term but leave me less prepared for exams where I have to solve problems independently. For homework, I plan to attempt problems myself and use my notes and class examples to aid me. I will try to identify the error and understanding its cause before looking for outside help. If I am still stuck, I can ask a TA, professor, or tutor for guidance to better understand the concept. My goal is to make sure I can explain the solution myself instead of only getting the code to pass the tests.
- e. false: the syllabus says that forgetting to push a file is not grounds for a regrade
- f. false: the syllabus says homework may be submitted late until the next exam for a 25% penalty

PROBLEM #2: Git

- a. can lead to errors: lab emphasizes to never clone one repo into another
- b. can lead to errors: the syncing system and Git can interfere with each other's file management
- c. is fine: cloning repositories at a higher-level location like cs104-repos folder is fine rather than inside a repo
- d. git add: tells git to track changes so they are ready for a commit
- e. git log -3: git log shows commit history, while -3 limits the outputs to the most recent 3 commits
- f. git clone git@github.com:usc-csci104-spring2026/resources.git

Name: Sophia Vinh-Reyes

Student ID#: 7334339093

HW # 1

PROBLEM #3: Runtime Analysis

a. 

```
void f1(int n)
{
    int i=2;
    while(i < n){
        /* do something that takes O(1) time */
        i = i*i;
    }
}
```

Starting from 2:

before loop:  $L = 2 \rightarrow 2^1$   
 Iteration #1:  $L = 4 \rightarrow 2^2$   
 Iteration #2:  $L = 16 \rightarrow 2^4$   
 Iteration #3:  $L = 256 \rightarrow 2^8$

} the exponents double

[ after k iterations: the loop stops when:  
 $L = 2^{2^k}$   $2^{2^k} \geq n$  ]

Solving for big- $\theta$ :

$$2^{2^k} \geq n$$

$$(2^{2^k} \geq n) \log_2$$

$$2^k \geq \log_2 n$$

$$(2^k \geq \log_2 n) \log_2$$

$$k \geq \log_2 (\log_2 n)$$

$$\boxed{\Theta(\log \log n)} \leftarrow \text{ANSWER}$$

b. 

```
void f2(int n)
{
    for(int i=1; i <= n; i++){  $\Theta(n)$ 
        if( (i % (int)sqrt(n)) == 0){  $\Theta(\sqrt{n})$ 
            for(int k=0; k < pow(i,3); k++){  $\Theta(i^3)$ 
                /* do something that takes O(1) time */  $\Theta(1)$ 
            }
        }
    }
}
```

$$\sum_{i=0}^n i^c = \Theta(n^{c+1})$$

$$\sum_{j=1}^M j^3 = \Theta(M^4), \text{ where } M = \sqrt{n}$$

$$\text{pow}(i,3) \rightarrow i^3$$

Inner loop runs when  $i$  is a multiple of approximately  $\sqrt{n}$ ↳ example: runs if  $i = \sqrt{n}, 2\sqrt{n}, 3\sqrt{n}, \dots$ let  $i = j\sqrt{n}$ , where  $j$  goes from 1 up to  $\sqrt{n}$ 

Work from inner loop:

$$\sum_{j=1}^{\sqrt{n}} \Theta(j\sqrt{n})^3 \rightarrow (\sqrt{n})^3 \sum_{j=1}^{\sqrt{n}} j^3 \rightarrow n^{3/2} \sum_{j=1}^{\sqrt{n}} j^3$$

$$\sum_{j=1}^{\sqrt{n}} j^3 = \Theta((\sqrt{n})^4) = \Theta(n^2)$$

$$\Theta(n^{3/2}) \cdot \Theta(n^2) = \Theta(n^{3.5})$$

work from outer loop:  $\Theta(n)$ 

$$\text{total: } \cancel{\Theta(n)} + \Theta(n^{3.5}) \Rightarrow \boxed{\Theta(n^{3.5})} \leftarrow \text{ANSWER}$$

Name: Sophia Vinh-Reyes

Student ID#: 7334339093

HW # 1

PROBLEM #3: Runtime Analysis

```

c. for(int i=1; i <= n; i++){  $\Theta(n)$ 
    for(int k=1; k <= n; k++){  $\Theta(n)$ 
        if( A[k] == i){
            for(int m=1; m <= n; m=m+m){  $\Theta(\log n)$ 
                // do something that takes  $O(1)$  time  $\Theta(1)$ 
                // Assume the contents of the A[] array are not changed
            }
        }
    }
}

```

Innermost for loop:

 $m = m + m$  $m = 2m \leftarrow$  doublesruns 1 up to  $n$ 

$$\sum_{m=1}^{\log n} \Theta(1) = \Theta(\log n)$$

If condition:

inner m loop only executes if condition is True

how often is it true?

 $\hookrightarrow$  outer loop  $k$  goes from 1 to  $n$  $\hookrightarrow A[k] == i$  can be true for at most one value of  $k$ 

$$[A[k] == i] = 1 \quad \leftarrow$$

formal summation:

$$\text{total time} = \sum_{i=1}^n \sum_{k=1}^n (\text{time inside } k \text{ loop})$$

takes  $\Theta(1)$  time to check if statement (constant time)if condition is true it runs m loop  $\rightarrow \Theta(\log n)$  time

$$\text{total time} = \sum_{i=1}^n \sum_{k=1}^n \Theta(1) + \sum_{i=1}^n \sum_{k=1}^n [A[k] == i] \cdot \Theta(\log n)$$

$$\sum_{i=1}^n \sum_{k=1}^n \Theta(1) = \Theta(n^2)$$

$$\sum_{k=1}^n \left( \sum_{i=1}^n [A[k] == i] \cdot \Theta(\log n) \right) \rightarrow \sum_{k=1}^n 1 \cdot \Theta(\log n) = \Theta(n \log n)$$

$$\text{total time} = \cancel{\Theta(n^2)} + \Theta(n \log n)$$

$$\text{total time} = \boxed{\Theta(n \log n)} \leftarrow \text{ANSWER}$$

Name: Sophia Vinh-Reyes

Student ID#: 7334339093

HW # 1

PROBLEM #3: Runtime Analysis

d.

```

int f (int n)
{
    int *a = new int [10];
    int size = 10;
    for (int i = 0; i < n; i++)  $\Theta(n)$ 
    {
        if (i == size)  $\leftarrow$  executes if list is full up
        {
            int newsize = 3*size/2; newsize = 1.5 (size)
            int *b = new int [newsize];
            for (int j = 0; j < size; j++) b[j] = a[j];  $\Theta(\text{size})$ 
            delete [] a;
            a = b;
            size = newsize;
        }
        a[i] = i*i;
    }
}

```

list capacities:

before if statement: 10

1<sup>st</sup> iteration: 15  $\rightarrow 10(1.5)$ 2<sup>nd</sup> iteration: 22  $\rightarrow 10(1.5)^2$ 3<sup>rd</sup> iteration: 33  $\rightarrow 10(1.5)^3$ 

every resize copies all elements from previous array

 $\hookrightarrow$  total work:  $10 + 15 + 22 + 33 + \dots + n$ total work:  $\Theta(n)$ outer loop:  $\Theta(n)$  $T(n) = \Theta(n) + \Theta(n) \Rightarrow \boxed{\Theta(n)} \leftarrow \text{ANSWER}$ ★ if list wasn't resizing  $\rightarrow O(1)$ resizing takes  $\Theta(\text{size})$  until it reaches  $\Theta(n)$  $\hookrightarrow$  because  $i$  is capped at  $n$

Name: Sophia Vinh-Reyes

Student ID#: 7334339093

HW # 1

PROBLEM #4: Linked List Recursion Tracing

```

struct Node {
    int val;
    Node* next;
};

Node* llrec(Node* in1, Node* in2)
{
    if(in1 == nullptr) {
        return in2;
    }
    else if(in2 == nullptr) {
        return in1;
    }
    else {
        in1->next = llrec(in2, in1->next);
        return in1;
    }
}

```

a. in1: 1 → 2 → 3 → 4 → NULL

in2: 5 → 6 → NULL

in1 in2

Iteration #1: llrec(1, 5)

Iteration #2: llrec(5, 2)

Iteration #3: llrec(2, 6)

Iteration #4: llrec(6, 3)

Iteration #5: llrec(3, nullptr)

unwind iterations

in1: 1 → 5 → 2 → 6 → 3 → 4

in1: 5 → 2 → 6 → 3 → 4

in1: 2 → 6 → 3 → 4

in1: 6 → 3 → 4

in1: 3 → 4 → NULL

When llrec is called, 1 5 2 6 3 4 is returned

b. call-tree:

llrec(1, 5)

+ 1.next = llrec(5, 2)

+ 5.next = llrec(2, 6)

+ 2.next = llrec(6, 3)

+ 6.next = llrec(3, NULL)

unwinding:

llrec(3, NULL) returns: 3 → 4

+ returns 3

+ 6.next = 6 → 3 → 4

+ 2.next = 2 → 6 → 3 → 4

+ 5.next = 5 → 2 → 6 → 3 → 4

+ 1.next = 1 → 5 → 2 → 6 → 3 → 4

c. in1 = nullptr

in2 = 2

in1 in2

Iteration #1: llrec(nullptr, 2)

↳ if (nullptr == nullptr)

return in2

When llrec is called, 2 is returned